LIMITATIONS OF THE OBJECT TYPE

In our earlier email example, the `attachments` field was declared as an `object` type. While we created an email with just one attachment object, nothing stops us from creating multiple attachments (emails often have multiple files attached), as shown in the next listing.

Listing 4.27 Indexing a document with multiple attachments

```
PUT emails/_doc/2
{
  "to:":"mrs.doe@johndoe.com",
  "subject":"Multi attachments test",
  "attachments":[{
    "filename":"file2.txt",
    "filetype":"confidential"
  },{
    "filename":"file3.txt",
    "filetype":"private"
  }]
}
```

By default, the `attachments` field is the `object` type: an inner object composed of an array of attachment files. Notice that the classification type of the file file2.txt is `confidential` and of file3.txt is `private` (see table 4.4).

Table 4.4 Attachment filenames and classified types

Attachment name	File type
file2.txt	`confidential`
file3.txt	`private`

Our email document is indexed and has ID 1 and a couple of attachments. Let's work through a simple search requirement: matching documents given the filename file2.txt and file type `private`. Looking at the data in table 4.4, this query should return no results because file2.txt's classification is `confidential`, not `private`. Let's query and check the result.

To do so, we need to use an advanced query called a *compound query* that combines various leaf queries to create a complex query. One such compound query is the `bool search` query. Without going into detail about how the `bool` query is constructed, let's look at it in action. We write our `bool` query with two other query clauses:

- A `must` clause checking for all documents that match the attachment's filename using a `term` query
- A second `must` clause checking whether the file classification is `private`

The query is given in the following listing.